

PROGRAMMERINGSFÖRELÄSNING 18

Föreläsning 18

Ändrad 7 Oct 11:16

Kopiera fält

Man kan inte kopiera ett fält genom att skriva:

```
x = y
```

Det enda som händer är att pekarvariabeln `x` får en ny adress som värde. Nej, om vi vill kopiera ett fält måste vi kopiera varje element för sig. Det gör min funktion `copy()`:

```
1  #include <stdio.h>
2
3  void copy(int length, float x[], float y[])
4  {
5      int i;
6      for (i = 0; i < length; i++)
7          x[i] = y[i];
8  }
9
10
11 void print(char *text, int length, float x[])
12 {
13     int i;
14     printf("%s", text);
15     printf("{");
16     for (i = 0; i < length-1; i++)
17         printf("%g, ", x[i]);
18     if (length > 0)
19         printf("%g", x[i]);
20     printf("}\n");
21 }
22
23 void main()
24 {
25     int i, length = 5;
26     float x[5] = {1, 2, 3, 4, 5};
27     float y[5] = {2.2, 2.2, 2.2, 2.2, 2.2};
28
29     print("x (length = 0) = ", 0, x);
30     print("x (length = 1) = ", 1, x);
31     print("x = ", length, x);
32     print("y = ", length, y);
33
34     copy(length, y, x);
35
36     print("x = ", length, x);
37     print("y = ", length, y);
38 }
```

■ Jag använder `copy()` för att kopiera från `x` till `y` (rad 34). Längden på fälten är parameter till `copy()`.

■ Utskriften i programmet kommer från funktionen `print()`. Första argumentet är en sträng som skrivs ut som den är (rad 14).

`print()` skriver sedan ut ett `float`-fält. Funktionen använder samma form som används i initieringar av fält. Kommatecknen ställer till lite problem. Likaså det faktum att `length` kan vara noll.

(Allt ska alltid fungera för specialfallet noll.
 Ni har väl skrivit laboration 3 så att det är möjligt att
 kasta noll tärningar?)

I min provkörning testar jag bland annat att `print()` fungerar för `length` noll och ett:

```
1 x (length = 0) = {}
2 x (length = 1) = {1}
3 x = {1, 2, 3, 4, 5}
4 y = {2.2, 2.2, 2.2, 2.2, 2.2}
5 x = {1, 2, 3, 4, 5}
6 y = {1, 2, 3, 4, 5}
```

Avgöra likhet mellan fält

Vi kan inte avgöra om två fält är lika, genom att skriva

```
if (x == y) ...
```

Det som testas är bara om `x` och `y` pekar på samma adress. Nej, istället måste man jämföra element för element.

Jag har skrivit fem(!) olika versioner av en funktion som avgör likhet mellan två `double`-fält.

Gör en egen, och kolla sedan vilken den blev mest lik!

Version nummer 1:

```
3 int equal1(int length, double x[], double y[])
4 {
5     int i;
6     for (i = 0; i < length; i++)
7         if (x[i] != y[i])
8             return 0;
9     return 1;
10 }
```

Version nummer 2:

```
13 int equal2(int length, double x[], double y[])
14 {
15     int i, equal_so_far = 1;
16     for (i = 0; i < length; i++)
17         if (x[i] != y[i])
18             equal_so_far = 0;
19     return equal_so_far;
20 }
```

Version nummer 3:

```
23 int equal3(int length, double x[], double y[])
24 {
25     int i, eq = 1;
26     for (i = 0; i < length; i++)
27         eq = eq && x[i] == y[i];
28     return eq;
29 }
```

Version nummer 4:

```
32 int equal4(int length, double x[], double y[])
33 {
34     int i = 0;
```

```

35     while (i < length && x[i] == y[i])
36         i++;
37     return i == length;
38 }

```

Version nummer 5:

```

41 int equal5(int length, double x[], double y[])
42 {
43     int i = 0;
44     while (i < length && *x++ == *y++)
45         i++;
46     return i == length;
47 }

```

Sök upp-byt ut

I er textbehandlare har ni säkert funktionen "sök upp--byt ut". Jag har skrivit en funktion som utför detta.

Utskriften skapar illusionen av en initierad debatt:

```

1  Ändra från: Ada
2  Till: FORTRAN
3  Det är Ada som är bäst.
4  Det är FORTRAN som är bäst.
5  Ada, sa jag.
6  FORTRAN, sa jag.
7  Ada, Ada, Ada!
8  FORTRAN, FORTRAN, FORTRAN!
9  ^d

```

■ **Algoritm:** Anta att vi ska byta ut "Ada" mot "FORTRAN". Varje inmatningsrad gås igenom. Tecken för tecken. Normalt skrivs tecknet ut.

Men om tecknet är "A" och följande tecken är "da" (= "Ada"), då skrivs "FORTRAN" ut istället. Därefter hoppar vi över strängen "Ada", och fortsätter. [Här är en bild](#) som illustrerar tillvägagångssättet.

Närmare bestämt fungerar mitt program på följande sätt: Programmet jämför "Ada" med *hela* raden -- frågan ställs om raden *börjar på* "Ada". Vi stryker tecken från början av raden allteftersom.

Funktionen `prefix()` undersöker om återstoden av raden börjar på strängen "Ada".

Källkod:

```

1  #include <stdio.h>
2  #include <sys/types.h>
3  #include <strings.h>
4
5  #define MAXLINE 300
6  #define MAXWORD 30
7
8  int prefix(char s1[], char s2[])
9  {
10     char i = 0;
11     while (s1[i] == s2[i] && s1[i] != '\0' && s2[i] != '\0')
12         i++;
13     return s1[i] == '\0';
14 }
15
16 void main()
17 {
18     char w1[MAXWORD], w2[MAXWORD];
19     char s[MAXLINE];
20     int length_w1, i;
21
22     printf("Ändra från: ");
23     gets(w1);

```

```

24     printf("Till: ");
25     gets(w2);
26
27     length_w1 = strlen(w1);
28
29     while (gets(s) != NULL) {
30         i = 0;
31         while (s[i] != '\0')
32             if (prefix(w1, &s[i])) {
33                 printf("%s", w2);
34                 i = i + length_w1;
35             } else
36                 putchar(s[i++]);
37         putchar('\n');
38     }
39 }

```

■ Funktionen `prefix()` (rad 8--14) avgör om `s2` börjar på `s1`:

`prefix("AB", "ABC")` är sant eftersom "ABC" börjar på "AB".

`prefix("BC", "ABC")` är falskt.

`prefix("ABD", "ABC")` är falskt.

`prefix("ABCD", "ABC")` är falskt.

`prefix("", "ABC")` brukar räknas som sant.

`prefix("", "")` brukar också räknas som sant.

I `prefix()` jämförs tecken för tecken i `s1` och `s2`. Loopen avbryts när tecknen skiljer sig åt. Alternativt om någon av strängarna tar slut.

Det enda som görs i loopkroppen är att index `i` ökas med ett (rad 12).

Returvärde är `s1[i] == '\0'` (rad 13). För om loopen fortsätter tills `s1` är slut, så måste `s1` vara ett prefix till `s2`.

■ På raderna 22--25 läser vi in strängen som ska ersättas -- `w1` -- och dess ersättare -- `w2`.

■ Rad 27 anropar funktionen `strlen()` som returnerar antalet tecken i `w1`. När vi hittat "Ada" ska vi hoppa över dessa tre tecken (se rad 34).

Funktionen `strlen()` finns (på mitt system) i `strings.h`. I mitt system måste man dessutom inkludera `sys/types.h` för att kunna använda `strings.h`. Jag har inte tagit reda på varför.

■ I huvudloopen i huvudprogrammet (rad 29--38) läser vi in hela nästa inmatningsrad till variabeln `s`.

Huvudloopen fortsätter tills `gets()` returnerar `NULL`.

Funktionen `gets()` returnerar värdet `NULL` (= adress 0) när nåt gått fel. Det som gått fel är att indata har tagit slut -- att användaren tryckt *[Control]+[d]*.

Funktioner som returnerar adresser, returnerar som regel adress 0 (makrot `NULL`) när nåt gått fel.

■ Den inre loopen (rad 31--36) stegar igenom samtliga tecken i strängen `s`.

Anropet till `prefix()` är ganska finurligt. Vi skickar dels `w1`, och dels adressen till elementet `s[i]`. Vi lurar därmed `prefix()` att tro att en sträng ligger lagrad med början på adress `&s[i]`. Strängen sträcker sig till `'\0'`, dvs till slutet av den inlästa raden.

Det är samma typ av manipulation som i det första exemplet i [föreläsning 17](#).

Funktionen `prefix()` kommer att anropas med kortare och kortare sträng eftersom `i` ökas med ett för varje varv i loopen på rad 36

Rättare sagt: ibland ökas inte `i` med ett, utan med hela längden av `w1`. Detta sker om `prefix()` hittade `w1` på raden. Då skrivs `w2` ut.

■ Alltsammans kan också göras med ett [flitigare utnyttjande av pekare](#).

Övning

Denna version byter ut "Ada" mot "FORTRAN" även om "Ada" ingår i ett annat ord. Exempelvis byts "Adam" mot "FORTRANm".

Gör om programmet så att endast hela ord bytes ut!

Ledning: Skriv en egen funktion för inläsningen. Den ska läsa ett ord i taget. Och `prefix()` måste ersättas med en funktion som testat likhet.

Nästa lektion

